

IoT Intrusion Detection Using AI Techniques - Proposal

MEMBER: Mohid Nadeem

Introduction:

There has been a rapid growth of devices embedded with Internet of Things (IoT) programming, that has increased the importance of cybersecurity in modern digital systems. IoT devices such as smart watches, smart cameras, routers, sensors, and smart home systems are connected to networks and exchange large amounts of data. However, the increasing use of IoT systems has also created opportunities for cybercriminals to launch different types of network attacks.

Intrusion attacks including Distributed Denial of Service (DDoS), brute force attacks, malware-based attacks, and spoofing can affect the security, privacy, and availability of IoT systems. Traditional security systems often struggle to detect modern attacks because cyber threats can be more complex and large-scale.

Artificial Intelligence (AI) and Machine Learning (ML) techniques can help improve intrusion detection systems by automatically identifying suspicious patterns and abnormal network behaviour. This coursework aims to develop AI-based intrusion detection models using a publicly available IoT intrusion detection dataset. Different AI techniques will be implemented and evaluated to classify network traffic into different attack categories.

The selected problem represents a real-world cybersecurity problem because IoT security is becoming increasingly important in industries, businesses, healthcare systems, and smart environments. The coursework will demonstrate how AI techniques can support intelligent intrusion detection and improve network security.

Dataset Description:

The dataset selected for this coursework is the **CICIoT2023 Intrusion Detection Dataset** available at <https://www.kaggle.com/datasets/subhajournal/iotintrusion>. The dataset was introduced for research related to large-scale IoT cyberattacks and intrusion detection systems.

The dataset contains:

- Around 1 million network traffic instances
- 47 extracted network traffic features
- A label column representing different intrusion categories

The dataset includes different categories of IoT intrusion attacks (classes) including:

- DDoS
- DoS
- Brute Force
- Spoofing
- Recon
- Web-based attacks
- Mirai attacks

The dataset also contains multiple subcategories for several intrusion types, making it suitable for multiclass classification tasks.

The selected dataset satisfies the coursework requirements as:

- It has more than 100 samples
- It includes more than four features
- It contains multiple attack classes for classification

Important Features Used:

Although the dataset contains 47 features, not all features will be used directly during model training. Some unnecessary features may be removed during preprocessing to improve model performance. Some of important features include:

Feature	Description
flow_duration	Total duration of the network flow
Header_Length	Length of the packet header
Protocol	Communication protocol used in the traffic
Rate	Transmission rate of packets
Srate	Source packet rate
Drate	Destination packet rate
syn_flag_number	Number of SYN flags in packets
ack_flag_number	Number of ACK flags in packets
rst_flag_number	Number of RESET flags in packets
HTTP, HTTPS, DNS	Indicates protocol traffic

TCP, UDP, ICMP	Indicates protocol usage
Tot sum	Total packet-related values
Min	Minimum packet value
Max	Maximum packet value
AVG	Average packet value
Std	Standard deviation of traffic values
Tot size	Total size of packets
Variance	Variance in traffic behaviour
Weight	Weight value associated with traffic flow

These features are important because they help describe network traffic behaviour and can assist AI models in identifying suspicious activities and attack patterns.

Intrusion Classes

These classes represent different real-world cybersecurity threats commonly found in IoT environments.

Class	Description
DDoS	Large-scale attack that floods a network or server with traffic
DoS	Attack that attempts to make services unavailable
Brute Force	Attack that repeatedly tries passwords or credentials
Spoofing	Attack where identity or address information is falsified
Recon	Attack used to gather information about systems or networks
Web-based	Attack targeting websites or web applications
Mirai	Malware-based IoT botnet attack

Dataset Reduction

The original dataset contains approximately 1 million instances, which is very large for model training and experimentation for the coursework scope.

Therefore, a balanced subset of the dataset will be created during preprocessing. The reduced dataset will contain selected samples from each intrusion class to:

- reduce computational complexity
- reduce memory usage
- allow faster experimentation and evaluation

Proposed AI Techniques

Multilayer Perceptron

A Multilayer Perceptron neural network will be implemented using TensorFlow and Keras libraries. The model will help classify network traffic into different intrusion categories using extracted traffic features.

Ensemble Learning Model

A Random Forest ensemble model will be implemented using the Scikit-learn library. Random Forest will allow combining multiple decision trees to improve the classification accuracy. It can also help reducing overfitting.

Convolutional Neural Network

Although CNNs are mainly designed for image processing, this coursework scope includes the use of CNNs to reshape network traffic data for intrusion detection. The CNN model will help compare deep learning performance against machine learning approach.

Clustering Technique

K-Means clustering algorithm will be implemented to classify network traffic based on similarities in traffic behaviour and attack patterns. The clustering results will be compared with the actual intrusion classes to evaluate clustering accuracy and hidden traffic patterns.

Objectives:

1. To analyse IoT network traffic and intrusion attack behaviour using a real-world cybersecurity dataset.
2. To preprocess intrusion detection data for machine learning applications.
3. To implement an MLP classifier for IoT intrusion detection.
4. To develop and evaluate a Random Forest model for intrusion classification.
5. To implement a CNN model.
6. To apply clustering techniques for grouping similar intrusion patterns.
7. To evaluate and compare the performance of different AI techniques for intrusion detection.

Report

IoT Intrusion Detection Using AI Techniques

MEMBER: Mohid Nadeem

Contents

1. Dataset Description:.....	8
2. Data Pre-processing and Visualisation	8
2.1. Dataset Inspection	8
2.2. Dataset Reduction and Balancing	9
2.3. Missing Values and Duplicate Analysis.....	10
2.4. Data Type Inspection	10
2.5. Visualisation and Exploratory Data Analysis	11
2.6. Feature Selection and Importance.....	Error! Bookmark not defined.
2.7. Label Encoding	14
2.8. Feature Scaling	15
2.9. Dataset Limitations	15
3. Task A – Multilayer Perceptron (MLP) Classifier	16
3.1. Initial MLP Implementation	16
3.2. Initial Model Performance	16
3.3. Model Improvement and Tuning	17
3.4. Improved Model Performance	17
3.5. Training and Validation Analysis	18
3.6. Classification Report Findings	19
3.7. Confusion Matrix Analysis.....	19
4. Task B – Random Forest Ensemble Classifier	20
4.1. Random Forest Implementation	20
4.2. Model Performance	20
4.3. Confusion Matrix Analysis.....	21
4.4. Findings and Observations.....	22
4.5. Comparison with Task A (MLP)	22

5. Task C – Convolutional Neural Network (CNN)	23
5.1. CNN Implementation	23
5.2. CNN Architecture.....	23
5.3. Model Performance	24
5.4. Training and Validation Analysis	24
5.5. Classification Report and Confusion Matrix Analysis	25
5.6. Comparison with Task A (MLP) and Task B (Random Forest).....	26
6. Task D – K-Means Clustering.....	26
6.1. Implementation	26
6.2. Clustering Process	26
6.3. Silhouette Score Evaluation	27
6.4. PCA-Based Cluster Visualisation.....	27
6.5. Cluster Distribution Analysis.....	28
6.6. Comparison with Supervised Learning Models	28
7. Ethical and Social Impact of AI Solutions	29

1. Dataset Description:

As mentioned in the proposal, the dataset selected for this coursework is the **CICIoT2023 Intrusion Detection Dataset** available at <https://www.kaggle.com/datasets/subhajournal/iotintrusion>.

The dataset contains:

- Approximately 1 million network traffic instances
- 47 extracted network traffic features
- A label column for different intrusion categories

The dataset includes different categories of IoT intrusion attacks (classes) including:

- DDoS
- DoS
- Brute Force
- Spoofing
- Recon
- Web-based attacks
- Mirai attacks

2. Data Pre-processing and Visualisation

2.1. Dataset Inspection

Python Jupyter notebook has been used for the implementation of all the tasks. The CICIoT2023 intrusion detection dataset was read via Pandas library. Initial inspection of the dataset was performed using functions including `head()`, `shape()`, `info()`, and `value_counts()` in order to understand the structure of the data, total number of records, available features, and attack categories.

The original dataset contained approximately 1 million network traffic records with 47 features and one label column representing different intrusion categories. The inspection process showed that the dataset included multiple IoT attack classes and protocol-related traffic features suitable for machine learning classification tasks.

```
[11]: df.shape
```

```
[11]: (1048575, 47)
```


2.2. Dataset Reduction and Balancing

Due to the very large size of the original dataset, a reduced balanced subset was created before model training. Seven major classes were selected from the dataset:

- BenignTraffic
- DDoS-ICMP_Flood
- DDoS-TCP_Flood
- DDoS-UDP_Flood
- DoS-TCP_Flood
- DoS-UDP_Flood
- Mirai-greeth_flood

A total of 3000 samples were randomly selected from each class, resulting in a balanced dataset of approximately 21,000 records. The balanced dataset also allowed fair comparison between different AI models during evaluation. The following figure shows the script that was run to get the sample portion from the complete dataset:

```
# List of selected major attack classes (reducing num of classes for cwk scope)
selected_classes = [
    'DDoS-ICMP_Flood',
    'DDoS-UDP_Flood',
    'DDoS-TCP_Flood',
    'DoS-UDP_Flood',
    'DoS-TCP_Flood',
    'BenignTraffic',
    'Mirai-greeth_flood'
]

# Creating an empty dataframe
# will store the reduced balanced dataset
reduced_df = pd.DataFrame()

# Loop through each selected class
for cls in selected_classes:

    # Filter rows belonging to current class
    # And randomly select 3000 samples
    sample = df[df['label'] == cls].sample(n=3000, random_state=42)

    # Add sampled rows into reduced dataset
    reduced_df = pd.concat([reduced_df, sample])
```

2.3. Missing Values and Duplicate Analysis

The dataset was checked for missing values using the `isnull()` function. No missing values were found in any feature columns. Therefore, no imputation or replacement techniques were required during preprocessing.

The figure below is a screenshot that depicts the output for missing values against some features. Other features not included in the figure have the same results.

```
flow_duration      0
Header_Length      0
Protocol Type      0
Duration           0
Rate              0
Srate             0
Drate            0
fin_flag_number    0
syn_flag_number    0
rst_flag_number    0
psh_flag_number    0
ack_flag_number    0
ece_flag_number    0
cwr_flag_number    0
ack_count          0
syn_count          0
fin_count          0
urg_count          0
rst count          0
```

The dataset was also checked for duplicate records. A small number of duplicate rows (count: 28) were identified and removed from the reduced dataset to improve data quality and avoid repeated traffic patterns during model training. After duplicate removal, the final dataset contained 20,972 records.

```
Duplicate Rows: 28
```

2.4. Data Type Inspection

Data types of all features were inspected to ensure compatibility with machine learning models. Most network traffic features were stored as numeric values in either integer or float format. The label column contained categorical string values representing different intrusion classes.

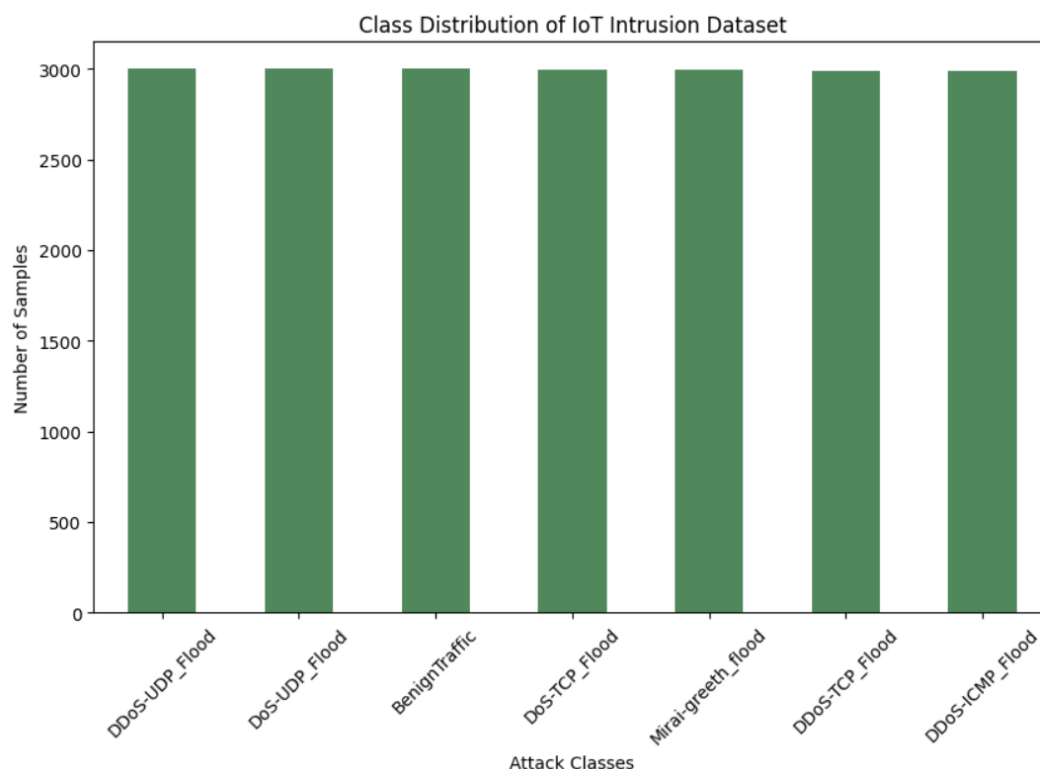
The inspection confirmed that the dataset structure was suitable for machine learning preprocessing and model training.

2.5. Visualisation and Exploratory Data Analysis

I applied four visualisation techniques to better understand the dataset and identify traffic behaviour patterns.

Class Distribution Analysis

A bar chart was generated to visualise the distribution of intrusion classes within the reduced dataset. The graph confirmed that all selected classes contained equal numbers of samples (3000 for each class) after balancing. This helps reduce classification bias during model training.

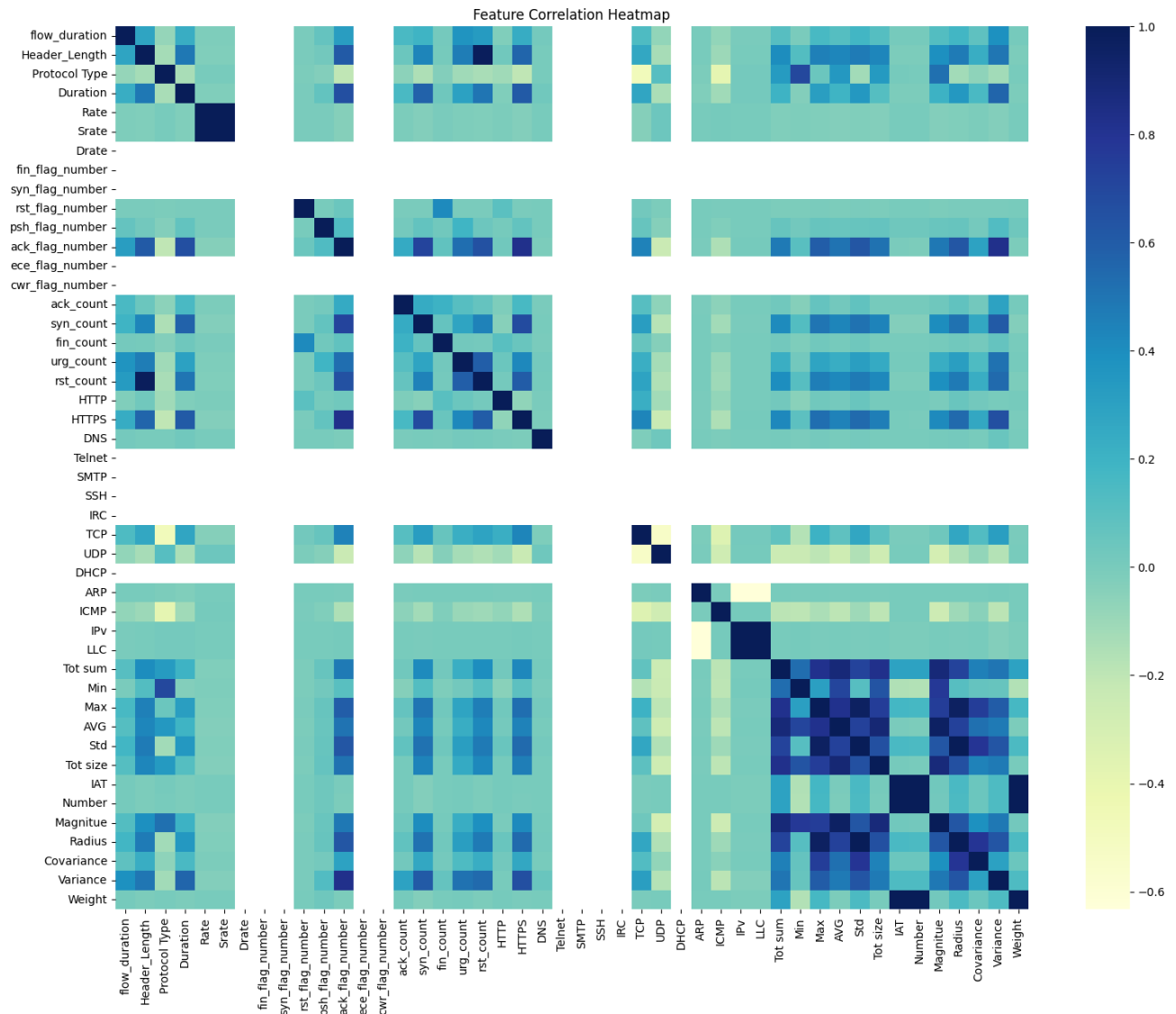


Findings:

- The reduced dataset is balanced successfully.
- Each selected intrusion class contains 3000 samples.
- Balanced classes help reduce model bias during training and evaluation.
- The dataset now provides fair representation for all selected attack categories.

Correlation Heatmap

A correlation heatmap was generated to analyse relationships between network traffic features. The visualisation showed that several statistical traffic features such as Total Sum, Average, Variance, Radius and Weight were strongly correlated with each other.



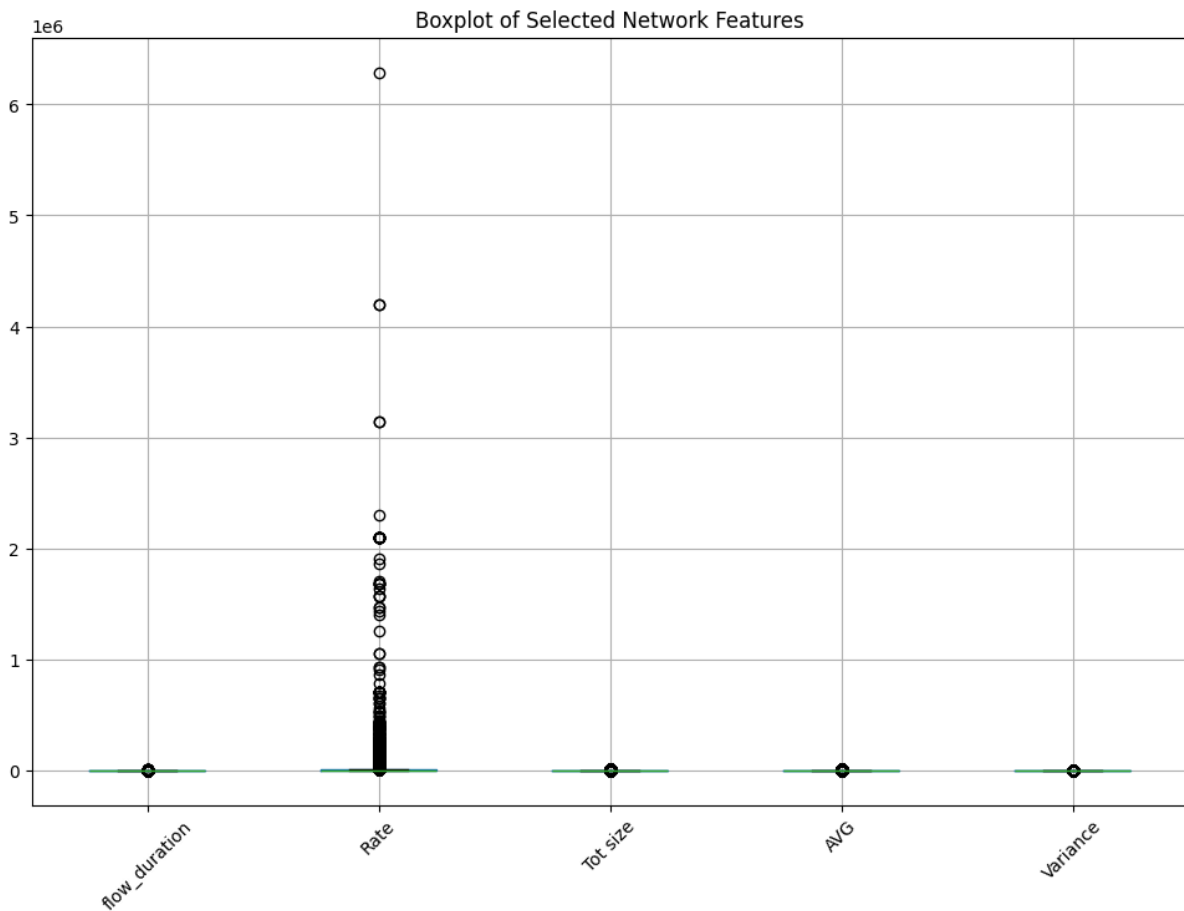
Findings:

- Several network traffic features show moderate to strong positive correlations.
- Protocol-related features show weaker correlations with statistical traffic features.
- The heatmap indicates that some features may contain overlapping information, which may influence model learning.

Outlier Analysis

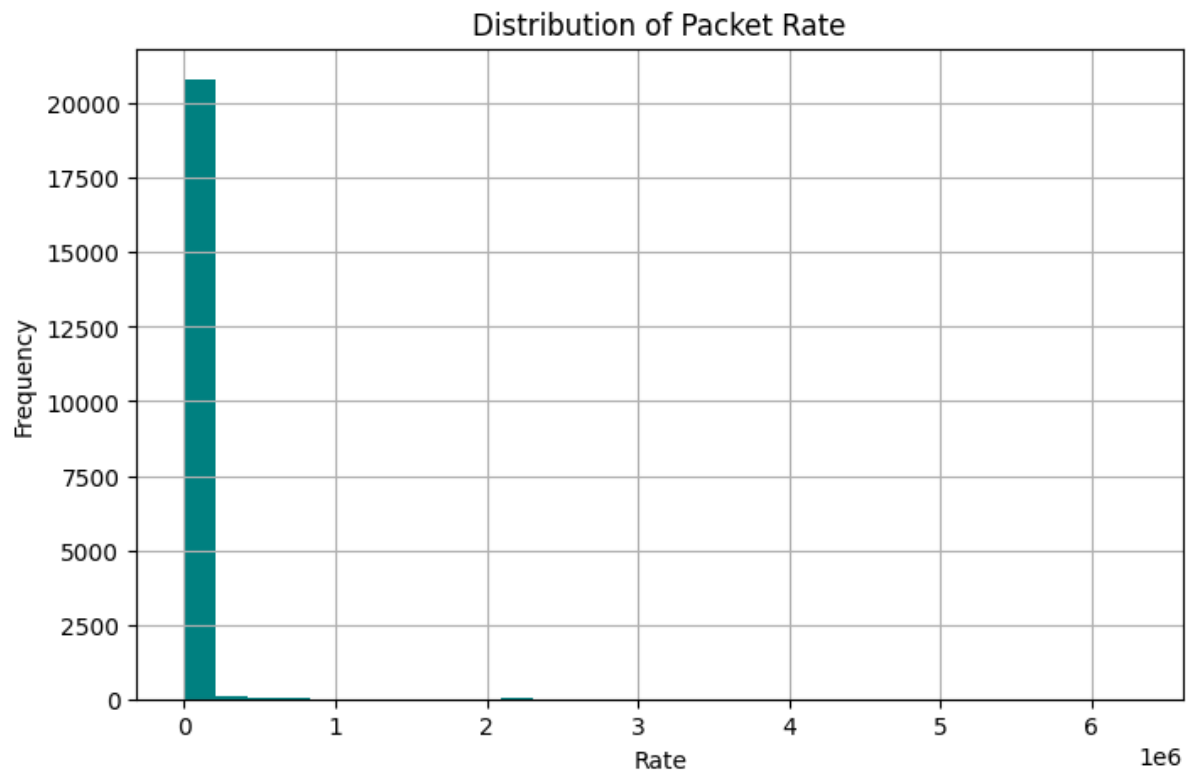
Boxplots were generated for selected traffic features such as flow_duration, rate, total size, average, and variance. Several outliers were identified, especially in the Rate feature.

These outliers were expected because intrusion attacks such as DDoS and flooding attacks generate unusually high traffic behaviour. Therefore, I decided not to remove outliers as they may represent meaningful attack patterns.



Feature Distribution Analysis

A histogram of the Rate feature was generated to observe traffic distribution. The graph showed a heavily right-skewed distribution where most traffic records contained lower packet rates while fewer records contained extremely high traffic values.



This behaviour can be considered common in intrusion detection datasets because attack traffic often generates sudden traffic spikes.

2.6. Label Encoding

The intrusion class labels were originally stored as text values. Since ML algorithms require numeric target values, label encoding was applied using the LabelEncoder function from Scikit-learn.

Each attack category has been converted into a unique numeric value from 0 to 6 while preserving the original class relationships.

2.7. Feature Scaling

Feature scaling was applied using the StandardScaler method from Scikit-learn. Scaling was necessary because several traffic-related features contained very different numeric ranges. For example, packet rate values could reach extremely large numbers, while protocol flags contained only binary values such as 0 or 1.

Without scaling, features with larger numeric ranges could dominate the training process of DL and clustering algorithms. Then, the dataset was split into training and testing sets for model development and evaluation. Training set is kept 80% of the total samples, ultimately leaving the other 20% for the testing set.

```
print(X_train_scaled.shape)
print(X_test_scaled.shape)

(16777, 46)
(4195, 46)
```

2.8. Dataset Limitations

One limitation of the dataset is its large size, which increases computational complexity and training time. A reduced balanced subset had to be created to make experimentation more manageable within the coursework scope.

Another limitation is that the dataset represents benchmark network traffic scenarios. Real-world IoT environments may contain more dynamic traffic behaviour and attack variations that are not fully represented in the dataset.

3. Task A – Multilayer Perceptron (MLP) Classifier

3.1. Initial MLP Implementation

I implemented a Multilayer Perceptron neural network using TensorFlow and Keras libraries for multiclass IoT intrusion classification. The model was developed using a Sequential architecture with four hidden layers containing:

- 1st with 15 neurons
- 2nd with 25 neurons
- 3rd with 25 neurons
- 4th with 20 neurons

For activation within hidden layers, the Rectified Linear Unit function was used, and for the output layer, the Softmax activation function was used.

The model was compiled using:

- Adam optimizer
- Categorical crossentropy loss function
- Accuracy as the evaluation metric

The dataset was trained using scaled training data and evaluated using the testing dataset.

3.2. Initial Model Performance

The first implementation of the MLP model produced weak classification performance with approximately 42% testing accuracy. The classification report showed that several classes were not being predicted correctly, and the confusion matrix indicated that the model was confusing certain attack categories.

The initial results showed that:

- The dataset contained complex traffic behaviour
- Several intrusion classes shared similar network characteristics
- The model required further tuning and regularisation

The model especially struggled to correctly classify several TCP-based DDoS and DoS attack categories.

3.3. Model Improvement and Tuning

To improve the performance of the MLP model, several modifications were applied during the tuning stage.

Increased Training Epochs

The number of training epochs was increased from 20 to 50. This allowed the model to learn deeper traffic behaviour patterns and improve convergence during training.

Dropout Regularisation

Added dropout layers with a dropout rate of 0.2 after hidden layers. Dropout regularisation can improve the model by disabling neurons randomly during training, resulting in the model to generalise better across unseen traffic patterns.

Batch Size Adjustment

The batch size was increased to 64 to improve training stability and learning efficiency during optimisation.

3.4. Improved Model Performance

After applying the tuning improvements, the MLP model achieved significantly better performance.

Metric	Result
Test Accuracy	77.57%
Test Loss	0.3673

The improved classification report showed strong performance for several attack classes including:

- BenignTraffic,
- DDoS-ICMP_Flood,
- Mirai-Greeth_Flood.

The model achieved precision and recall values close to 1.00 for these classes.

However, the model still showed moderate confusion for:

- DDoS-TCP_Flood
- DoS-TCP_Flood

This behaviour is understandable because both attack categories generate similar TCP-based network traffic behaviour.

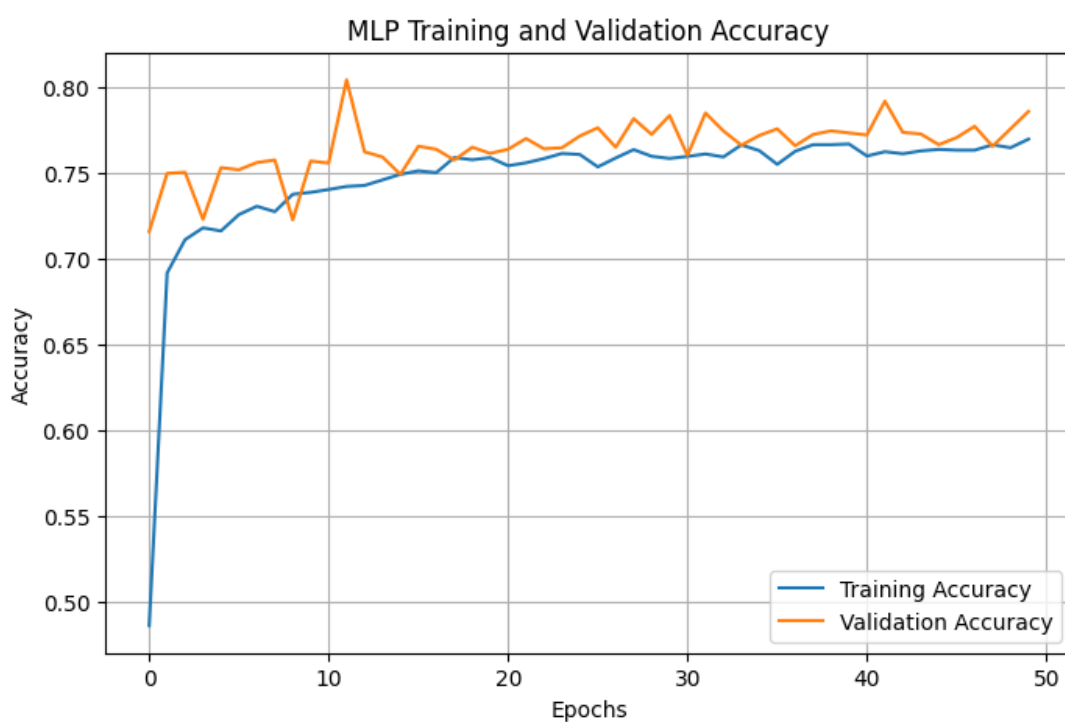
3.5. Training and Validation Analysis

Training and validation accuracy graphs were generated during the training process.

The visualisation showed:

- Gradual improvement in training accuracy
- Stable validation accuracy

Both training and validation accuracy curves persisted adjacent throughout the training process, indicating that the model generalised well. The model reached approximately 77–78% validation accuracy during later epochs.



3.6. Classification Report Findings

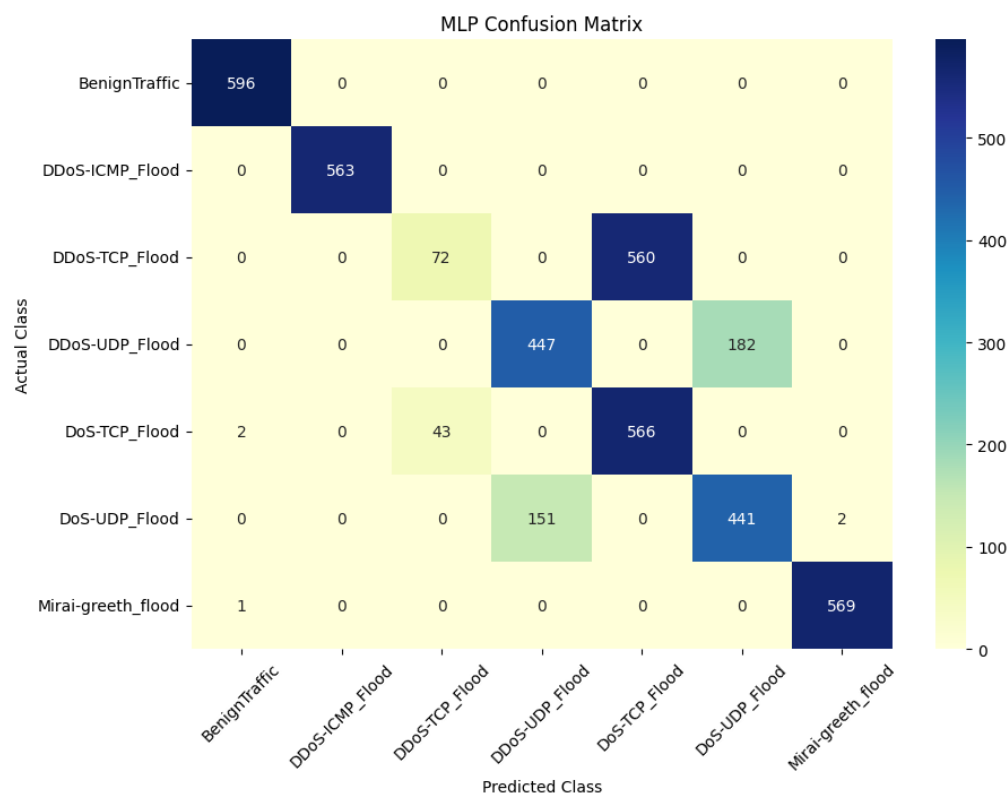
The classification report demonstrated that the MLP model proved to be effective in identifying some intrusion classes while showing moderate difficulty distinguishing closely related TCP-based flooding attacks.

The macro and weighted average scores indicated stable multiclass classification performance as depicted by the figure below:

	precision	recall	f1-score	support
BenignTraffic	0.99	1.00	1.00	596
DDoS-ICMP_Flood	1.00	1.00	1.00	563
DDoS-TCP_Flood	0.63	0.11	0.19	632
DDoS-UDP_Flood	0.75	0.71	0.73	629
DoS-TCP_Flood	0.50	0.93	0.65	611
DoS-UDP_Flood	0.71	0.74	0.72	594
Mirai-greeth_flood	1.00	1.00	1.00	570
accuracy			0.78	4195
macro avg	0.80	0.78	0.76	4195
weighted avg	0.79	0.78	0.75	4195

3.7. Confusion Matrix Analysis

Below is the generated confusion matrix that helps assess class-level prediction behaviour.



The confusion matrix showed that:

- BenignTraffic and Mirai attacks were classified very accurately
- DDoS-ICMP_Flood achieved almost perfect classification
- Some misclassification occurred between TCP-based DoS and DDoS attacks

4. Task B – Random Forest Ensemble Classifier

4.1. Random Forest Implementation

For Task B, I implemented the Random Forest algorithm using the Scikit-learn library. Random Forest is an ensemble classifier model, a ML technique that combines multiple decision trees to improve classification accuracy.

The model was trained using the pre-processed and scaled IoT intrusion dataset. 100 decision trees were added to the model. The model was trained using the training dataset and was evaluated through the the testing dataset.

4.2. Model Performance

The Random Forest model achieved extremely strong classification performance on the testing dataset. The test accuracy is 99.76%.

```
[50]: # Calculating the model accuracy

rf_accuracy = accuracy_score(y_test, y_pred_rf)
print("Random Forest Accuracy:", rf_accuracy)

Random Forest Accuracy: 0.9976162097735399
```

The classification report showed almost perfect precision, recall, and F1-scores for nearly all intrusion classes.

Most attack categories achieved

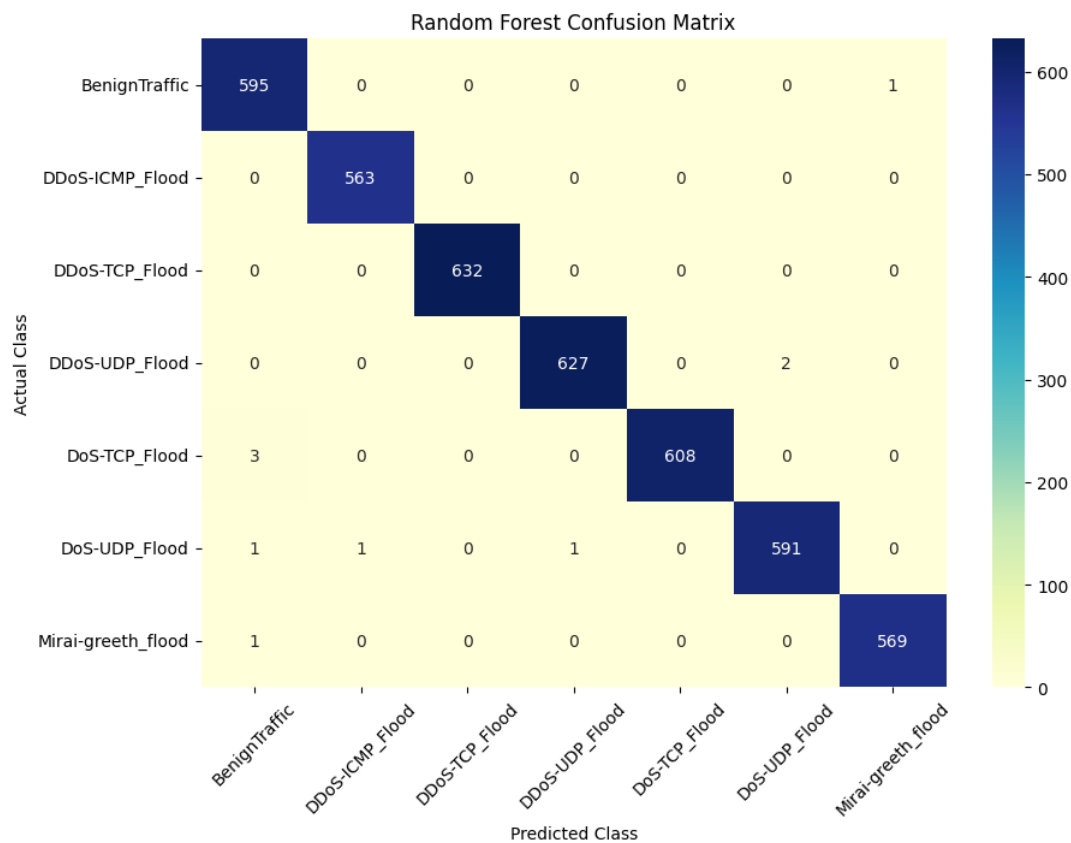
- Precision values close to 1.00
- Recall values close to 1.00
- F1-scores close to 1.00

The results show that the Random Forest model was highly effective in learning the network traffic behaviour patterns present within the dataset. The figure below shows the score values generated:

	precision	recall	f1-score	support
BenignTraffic	0.99	1.00	0.99	596
DDoS-ICMP_Flood	1.00	1.00	1.00	563
DDoS-TCP_Flood	1.00	1.00	1.00	632
DDoS-UDP_Flood	1.00	1.00	1.00	629
DoS-TCP_Flood	1.00	1.00	1.00	611
DoS-UDP_Flood	1.00	0.99	1.00	594
Mirai-greeth_flood	1.00	1.00	1.00	570
accuracy			1.00	4195
macro avg	1.00	1.00	1.00	4195
weighted avg	1.00	1.00	1.00	4195

4.3. Confusion Matrix Analysis

Only small levels of confusion were observed between certain DoS and DDoS attack categories, as in Task A. Compared to the MLP model, the Random Forest classifier produced fewer misclassifications.



4.4. Findings and Observations

The model achieved

- Very high classification accuracy
- Stable multiclass performance
- Low misclassification rates

The visualisations and confusion matrix also showed that the selected features were highly meaningful for identifying intrusion behaviour patterns.

4.5. Comparison with Task A (MLP)

The Random Forest model performed significantly better than the MLP neural network implemented in Task A.

Model	Accuracy
MLP Classifier	77.57%
Random Forest	99.76%

The MLP model showed difficulty distinguishing certain TCP-based DoS and DDoS attacks because several intrusion categories shared similar traffic characteristics.

In contrast, the Random Forest model was able to classify the network traffic patterns much more effectively. The ensemble structure of Random Forest allowed the model to learn complex feature interactions and decision boundaries more accurately.

The comparison also demonstrated that ensemble learning techniques can perform exceptionally well on intrusion detection datasets containing engineered network traffic features. Still, there may be some variance observed on increasing the sample size (as the sample size was greatly reduced while pre-processing to keep it effective for coursework scope)

5. Task C – Convolutional Neural Network (CNN)

5.1. CNN Implementation

For Task C, a CNN model was implemented. Although CNN models are commonly used for image-related tasks, they can also be applied to structured numerical datasets by learning local feature relationships and behaviour patterns.

Before training the CNN model, the scaled dataset was reshaped into a 3-D structure because CNN architectures require input in the form of:

- Samples
- Features
- Channels

The reshaped dataset was then used for model training and evaluation.

5.2. CNN Architecture

The CNN model was developed using:

- Two 1-D convolutional layers (ReLU activation function)
- Max pooling layers
- A flatten layer
- Dense fully connected layers (ReLU activation function)
- Dropout regularisation (to improve generalization performance)
- Output layer (Softmax activation function)

The model was trained using:

- Adam optimizer
- Categorical crossentropy loss function
- 30 training epochs
- Batch size of 64

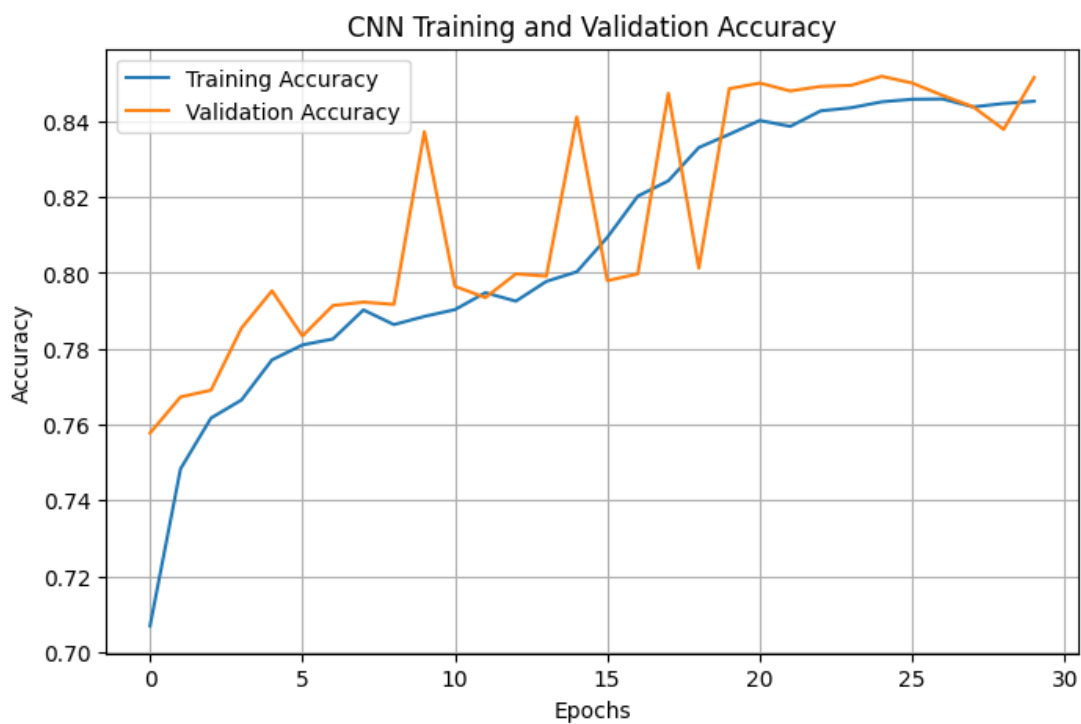
5.3. Model Performance

The CNN model achieved strong classification performance on the IoT intrusion dataset.

Metric	Result
Test Accuracy	84.70%
Test Loss	0.3160

5.4. Training and Validation Analysis

Both curves remained relatively close during training, indicating that the CNN model did not suffer from severe overfitting. The visualisations also determined that the CNN architecture learned traffic patterns more effectively than the earlier MLP implementation.

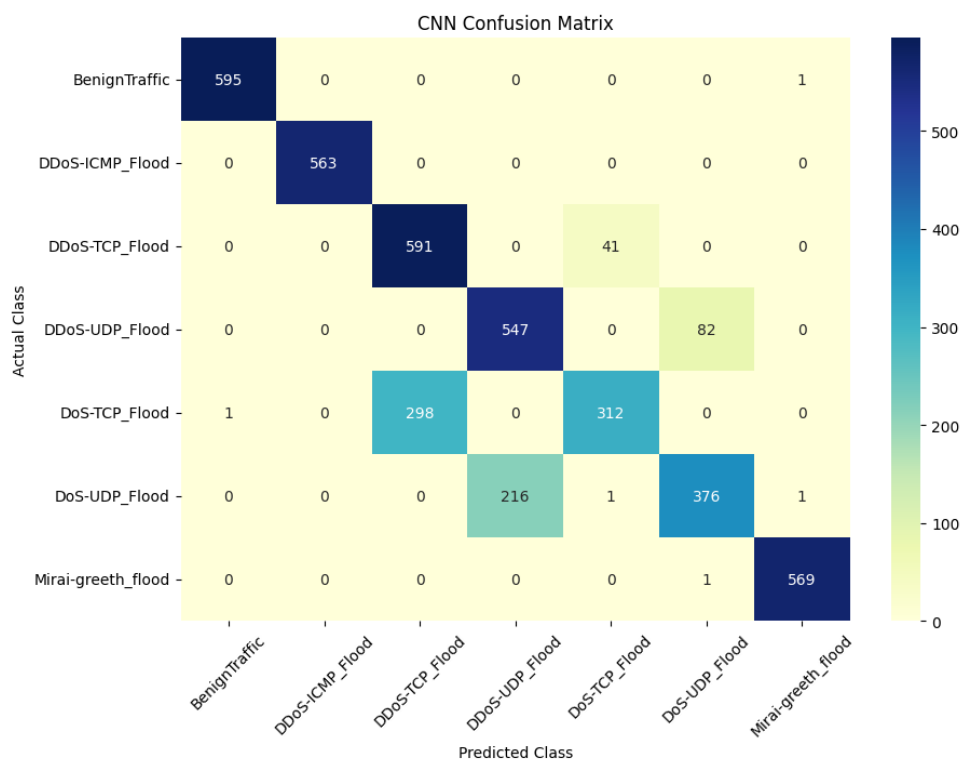


5.5. Classification Report and Confusion Matrix Analysis

Compared to the MLP model, the CNN classifier achieved higher precision values, and improved recall values and F1-scores. The CNN model improved classification performance for several attack categories that were previously difficult for the MLP model to distinguish correctly. Still, there was some moderate confusion found between DoS-TCP_Flood, DDoS-TCP_Flood, DoS-UDP_Flood, and DDoS-UDP_Flood.

	precision	recall	f1-score	support
BenignTraffic	1.00	1.00	1.00	596
DDoS-ICMP_Flood	1.00	1.00	1.00	563
DDoS-TCP_Flood	0.66	0.94	0.78	632
DDoS-UDP_Flood	0.72	0.87	0.79	629
DoS-TCP_Flood	0.88	0.51	0.65	611
DoS-UDP_Flood	0.82	0.63	0.71	594
Mirai-greeth_flood	1.00	1.00	1.00	570
accuracy			0.85	4195
macro avg	0.87	0.85	0.85	4195
weighted avg	0.86	0.85	0.84	4195

Most incorrect predictions occurred between TCP-based and UDP-based flooding attacks. This indicates that several attack categories share overlapping traffic characteristics, making complete separation more difficult even for deep learning models.



5.6. Comparison with Task A (MLP) and Task B (Random Forest)

The CNN model performed better than the MLP classifier implemented in Task A. The MLP model relied mainly on fully connected dense layers, while the CNN model was able to extract stronger local feature relationships using convolutional operations. As a result, the CNN model handled complex intrusion traffic behaviour more effectively and produced a little improved classification performance (with about 6% - 7% more accuracy).

Although the CNN model achieved improved performance, the Random Forest ensemble model from Task B still got the highest overall accuracy. This comparison suggests that traditional ensemble learning methods can outperform deep learning approaches when working with structured tabular datasets.

The Random Forest model was able to separate intrusion classes more effectively, while the CNN model still experienced moderate confusion between similar flooding attack categories.

CNN models may perform better than Random Forest when working with highly complex or sequential data such as images, videos, audio signals, or raw network packet streams.

6. Task D – K-Means Clustering

6.1. Implementation

For Task D, the Scikit-learn library is used to implement K-Means clustering algorithm. Unlike previous supervised learning models, K-Means clustering does not use class labels during training. Instead, the algorithm groups data instances based on similarities in feature behaviour.

The purpose of implementing K-Means clustering was to investigate whether network traffic belonging to different intrusion categories could naturally form meaningful groups based on their traffic characteristics.

6.2. Clustering Process

The clustering model was trained using the scaled IoT intrusion dataset. Since the dataset contained seven selected intrusion classes, the number of clusters was set to seven.

The K-Means algorithm was fitted on the scaled training dataset to identify cluster centroids and group similar network traffic instances together. After clustering, cluster labels were generated for the testing dataset. These cluster labels were then compared with the actual labels to analyse how effectively the clustering algorithm grouped similar attack behaviours.

6.3. Silhouette Score Evaluation

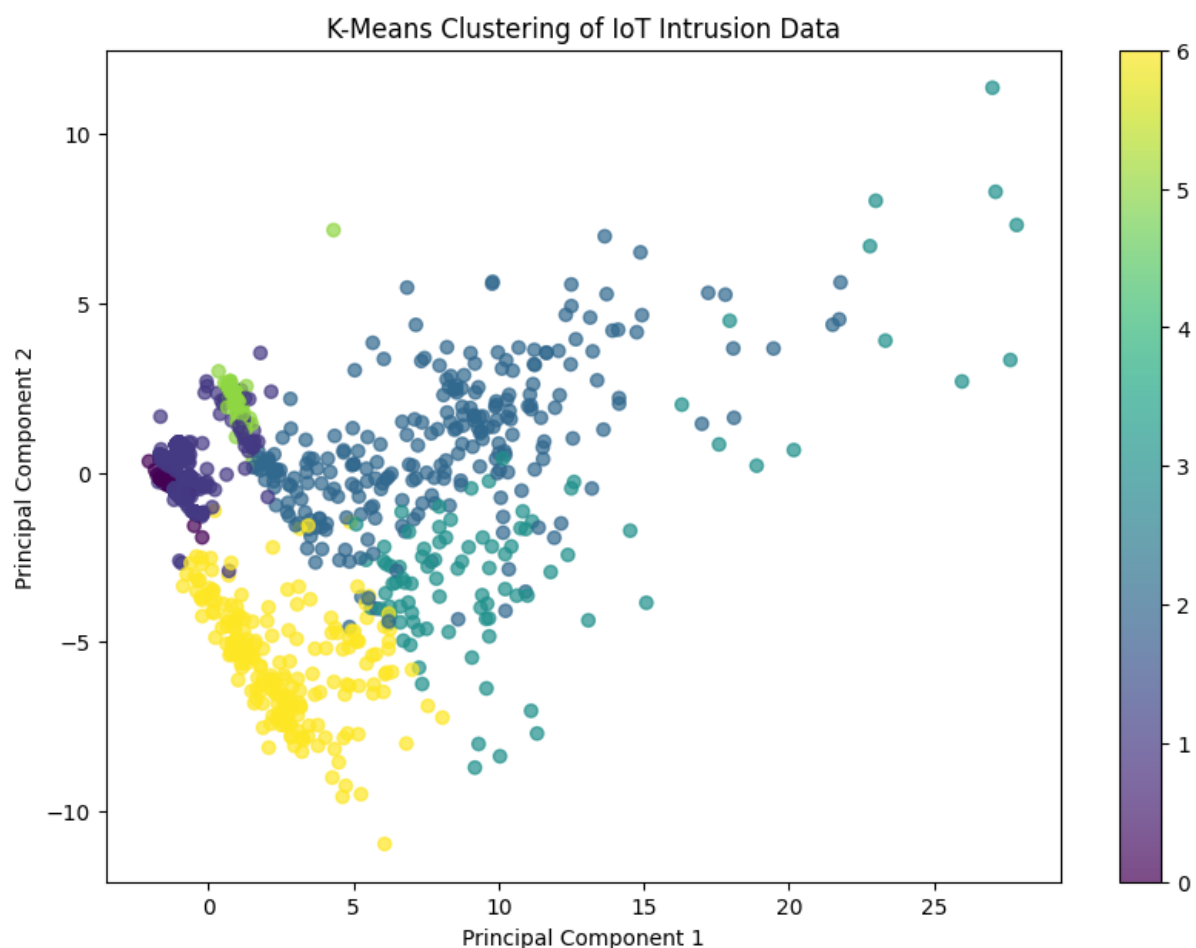
The clustering quality was evaluated using the Silhouette Score that was 0.473. The obtained score suggests that the clustering model achieved reasonably meaningful separation between intrusion traffic groups.

Considering the complexity and similarity between several flooding attack categories, this result indicates moderate to strong clustering performance for the intrusion detection dataset.

6.4. PCA-Based Cluster Visualisation

For visualization, Principal Component Analysis was used to reduce the dataset into a 2-D visual. The PCA scatter plot showed that several intrusion categories formed visibly separate traffic regions, while some attack categories partially overlapped.

The visualisation indicated that certain traffic behaviours formed compact and distinguishable clusters, while several flooding attack categories shared overlapping traffic characteristics.



6.5. Cluster Distribution Analysis

Cluster distribution analysis showed that some clusters contained significantly more samples than others.

This indicates that some intrusion categories share highly similar traffic behaviour, while other attack categories contain more unique network traffic patterns. The uneven cluster sizes demonstrate that the dataset contains varying levels of behavioural similarity across intrusion categories.

Cluster	
1	2500
5	569
0	565
2	257
6	209
3	95

6.6. Comparison with Supervised Learning Models

The clustering results if compared with the supervised learning approaches implemented in earlier tasks, then unlike MLP, CNN, and Random Forest, the K-Means algorithm performed clustering without using actual intrusion labels during training. Although K-Means successfully identified meaningful traffic groupings, its performance was lower than the supervised learning models because it relied only on feature similarity rather than labelled attack information.

The earlier supervised learning models achieved:

- MLP Accuracy: 77.57%
- CNN Accuracy: 84.70%
- Random Forest Accuracy: 99.76%

In comparison, K-Means clustering achieved moderate grouping performance with a silhouette score of 0.473.

7. Ethical and Social Impact of AI Solutions

Artificial Intelligence based intrusion detection systems can provide significant benefits for cybersecurity by helping organisations identify malicious network traffic quickly and reduce the impact of cyberattacks. In this coursework, AI models were used to classify different IoT intrusion categories such as DDoS, DoS, and Mirai attacks. Although these systems improve detection speed and automation, they also raise several ethical and social concerns that should be considered too.

One important ethical issue is **reliability and accountability**. If an AI-based intrusion detection system incorrectly classifies network traffic, it may either block legitimate users or fail to detect harmful attacks. False positives could interrupt important services, while false negatives may expose organisations to security breaches and financial losses. For this reason, AI systems used in cybersecurity should not operate without human monitoring and verification, especially in sensitive environments.

Another concern is **transparency**. Some AI models, like CNNs and MLPs, can behave like “black-box” systems where decisions are difficult to explain clearly. In cybersecurity environments, administrators may need understandable reasons behind attack detection decisions before taking action against network users or systems. This creates the need for explainable and interpretable AI solutions.

Privacy is also an important issue in intrusion detection systems. Network traffic data may contain sensitive information related to users, devices, or communication behaviour. Improper handling of this data could affect user privacy and trust. Therefore, organisations should ensure that collected traffic data is stored securely and used responsibly.

Bias and fairness should also be considered. If the training dataset does not properly represent real-world traffic conditions, the AI system may perform better on some attack types while performing poorly on others. This may lessen the effectiveness of the detection system in real deployment environments. Regular dataset updates and model evaluation are important to maintain fairness and reliability.

From a social perspective, AI-driven cybersecurity systems can positively impact organisations by reducing manual monitoring workload and improving response time against attacks. However, increasing automation may also reduce the need for certain human security monitoring tasks, potentially affecting some job roles within cybersecurity operations.

Overall, AI intrusion detection systems provide many advantages for improving cybersecurity, but ethical concerns such as accountability, transparency, privacy, fairness, and human oversight must be considered too to ensure responsible and trustworthy AI deployment.

References

GeeksforGeeks (2025) *MLP Classifier using TensorFlow and Keras*. Available at: <https://www.geeksforgeeks.org/deep-learning/multi-layer-perceptron-learning-in-tensorflow/> (Accessed: 24 May 2026).

GeeksforGeeks (2025) *Confusion Matrix in Machine Learning*. Available at: <https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/> (Accessed: 24 May 2026).

GeeksforGeeks (2025) *Convolutional Neural Network (CNN) Introduction*. Available at: <https://www.geeksforgeeks.org/introduction-convolution-neural-network/> (Accessed: 28 May 2026).

Scikit-learn (2026) *K-Means Clustering Documentation*. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html> (Accessed: 1 June 2026).

GeeksforGeeks (2025) *Random Forest Classifier using Scikit-learn*. Available at: <https://www.geeksforgeeks.org/random-forest-classifier-using-scikit-learn/> (Accessed: 2 June 2026).